Implementation Strategy

First, a 4-bit carry-lookahead adder is implemented and tested on a small scale. After verifying its correctness, two 4-bit CLA adders are connected in sequence: one handles the lower 4 bits and the other handles the upper 4 bits. The intermediate carry-out (c_4) is passed between them via a wire, forming a complete 8-bit carry-lookahead adder.

We implement the `cla_4bit` module first, which uses:

- Generate: $g[i] = a[i] \cdot b[i]$
- Propagate: $p[i] = a[i] \oplus b[i]$
- Carry lookahead equations:

$$
\begin{aligned}
c_1 &= g_0 + p_0 \cdot c_0 \\
c_2 &= g_1 + p_1 \cdot g_0 + p_1 \cdot p_0 \cdot c_0 \\
c_3 &= g_2 + p_2 \cdot g_1 + p_2 \cdot p_1 \cdot g_0 + p_2 \cdot p_1 \cdot p_0 \cdot c_0 \\
c_4 &= g_3 + p_3 \cdot g_2 + p_3 \cdot p_2 \cdot g_1 + p_3 \cdot p_2 \cdot p_1 \cdot g_0 + p_3 \cdot p_2 \cdot p_1 \cdot p_0 \cdot c_0
\end{aligned}
$$

Each sum bit is then computed as $s[i] = p[i] \oplus c[i]$.

The `cla_8bit` module connects two `cla_4bit` modules:

- The first processes a[3:0], b[3:0], and c_0 to output s[3:0] and c_4
- The second processes a[7:4], b[7:4], and c_4 to output s[7:4] and c_8

This modular approach improves readability, reusability, and hierarchical design.

Block Diagrams for Each Module

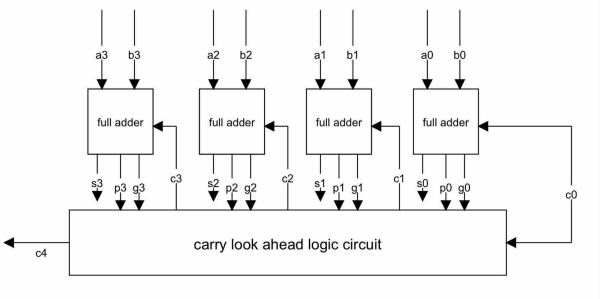

Figure 1: 4-bit CLA

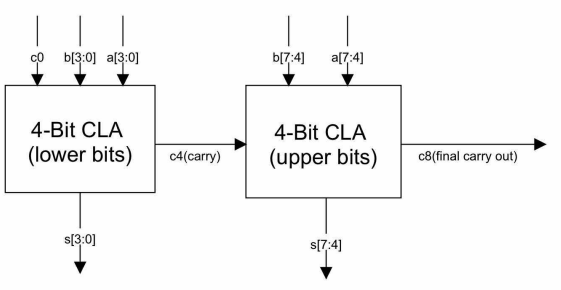

Figure 2: 8-bit CLA

Carry-Lookahead Theory and Justification

Ripple-Carry Adders (RCA) suffer from linear delay because each bit must wait for the previous carry. In contrast, the Carry Lookahead Adder (CLA) predicts carries in parallel, reducing critical path delay.

- Generate: $G_i = A_i \cdot B_i$
- Propagate: $P_i = A_i \oplus B_i$
- Carry-out: $C_{i+1} = G_i + P_i \cdot C_i$
- Sum:$S_i = P_i \oplus C_i$

This parallelism enables fast computation of carry signals across multiple bits. CLA is preferred in high-performance circuits due to this speed advantage.

Testbench Strategy

I wrote a Verilog testbench to validate the `cla_8bit` module. It includes edge and typical test cases:

- $1 + 1$ (basic case)
- $85 + 51$ (typical case)
- $255 + 1$ (overflow case, expect $s = 0$, $c_8 = 1$)
- $128 + 128 + 1$ (carry-in test, expect $s = 1$, $c_8 = 1$)

I use `$monitor` to display runtime values, and `$dumpfile`/`$dumpvars` to generate waveform data for GTKWave (or VScode extension WaveTrace).

Pseudocode of 4-bit CLA Logic

```
// Given: a[3:0], b[3:0], c0
// Output: s[3:0], c4

for i from 0 to 3:
   g[i] = a[i] & b[i]  // Generate
   p[i] = a[i] ^ b[i]  // Propagate

c[0] = c0
c[1] = g[0] | (p[0] & c[0])
c[2] = g[1] | (p[1] & g[0]) | (p[1] & p[0] & c[0])
c[3] = g[2] | (p[2] & g[1]) | (p[2] & p[1] & g[0]) | (p[2] & p[1] & p[0] & c[0])
c[4] = g[3] | (p[3] & g[2]) | (p[3] & p[2] & g[1]) |
     (p[3] & p[2] & p[1] & g[0]) | (p[3] & p[2] & p[1] & p[0] & c[0])
```

```
for i from 0 to 3:
   s[i] = p[i] ^ c[i]
```

Listing 1: Pseudocode for 4-bit CLA

Pseudocode of 8-bit CLA Construction

```
// Split inputs: a[7:0], b[7:0], c0
// Output: s[7:0], c8

// Lower 4-bit adder
[s[3:0], c4] = cla_4bit(a[3:0], b[3:0], c0)

// Upper 4-bit adder (uses c4 from lower)
[s[7:4], c8] = cla_4bit(a[7:4], b[7:4], c4)
```

Listing 2: Pseudocode for 8-bit CLA using 4-bit Modules

Pseudocode of Testbench Logic

```
// Initialize inputs
a = 0000_0000
b = 0000_0000
c0 = 0

// Apply test cases in sequence

// Test 1: 1 + 1
a = 0000_0001
b = 0000_0001
c0 = 0
// Expected: sum = 0000_0010, carry = 0

// Test 2: 85 + 51
a = 0101_0101 // binary of 85
b = 0011_0011 // binary of 51
c0 = 0
// Expected: sum = 1000_1000, carry = 0

// Test 3: 255 + 1
a = 1111_1111
b = 0000_0001
c0 = 0
// Expected: sum = 0000_0000, carry = 1

// Test 4: 128 + 128 + carry-in
a = 1000_0000
b = 1000_0000
c0 = 1
// Expected: sum = 0000_0001, carry = 1
```

Listing 3: Pseudocode for 8-bit CLA Testbench

Simulation Results Summary

Time (ns)	a	b	c_0	s	c_8
0	0	0	0	0	0
10	1	1	0	2	0
20	85	51	0	136	0
30	255	1	0	0	1
40	128	128	1	1	1

Observations:

- CLA correctly computes sums across all tested cases
- Overflow and carry-in are handled accurately

Waveform Screenshot

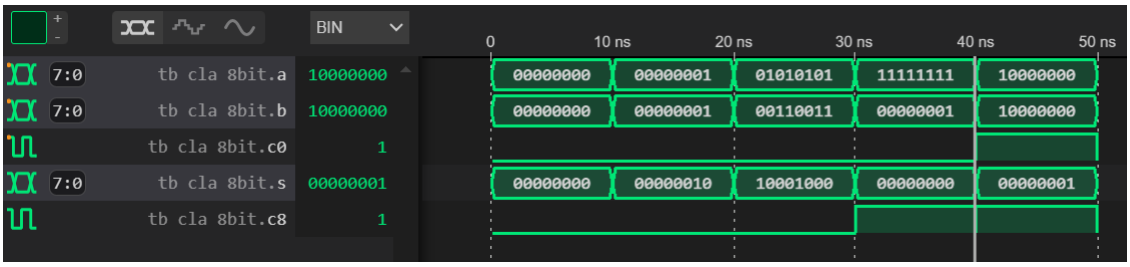

Figure 3: wave from VScode extension-WaveTrace

Include waveform image here showing transitions of `a`, `b`, `c0`, `s`, *and* `c8` *using WaveTrace.*

Conclusion

The modular 8-bit CLA design successfully demonstrates the performance advantage of hierarchical carry lookahead logic. Simulation confirms the accuracy of both sum and carry outputs.

This approach achieves a good balance between speed and modularity and can be scaled to wider word sizes using the same design principles.

GitHub Link

For more information and access to the Verilog source code, testbench, and report files, please visit the following GitHub repository:
https://github.com/sheng12077/logic_design-11330EECS101000-HW1